Efficient Hardware Implementation of the Lightweight CRYSTALS-Kyber

Trong-Hung Nguyen, *Graduate Student Member, IEEE*, Duc-Thuan Dam, *Graduate Student Member, IEEE*, Phuc-Phan Duong, *Graduate Student Member, IEEE*, Binh Kieu-Do-Nguyen, *Graduate Student Member, IEEE*, Cong-Kha Pham, *Senior Member, IEEE*, and Trong-Thuc Hoang, *Member, IEEE*

***Abstract*— Quantum computing raises questions about the security of data encrypted using modern methods. Hence, the National Institute of Standards and Technology (NIST) has undertaken standardization of post-quantum cryptography (PQC) algorithms to defend against attacks from both classical and quantum computers. Following four rounds of evaluation, CRYSTALS-Kyber has been selected for standardization. In this paper, we present an efficient hardware architecture of CRYSTALS-Kyber for resource-constrained IoT devices. Firstly, we propose a compact hash module for CRYSTALS-Kyber. A single buffer is designed to perform padding, hashing, and holding data. Hence, using large FIFOs for data input/output is eliminated. Then, we propose a novel non-memory-based iterative number theoretic transform (NMI-NTT) architecture. Finally, the data flow between modules is optimized to improve parallelization and execution time. Implementation results on an Artix-7 FPGA show that our design consumes minimal hardware resources compared to the designs reported to date, corresponding to 5487 LUTs, 3426 FFs, 1548 SLICEs, 3.5 BRAMs, and 2 DSPs. Our design computes key generation, encapsulation, and decapsulation phases in 3.3/4.5/6.1 K-cycles for Kyber512, 5.6/7.1/9.2 K-cycles for Kyber768, and 8.5/10.1/12.9 K-cycles for Kyber1024, with 185MHz operating frequency. Our area-time-product (ATP) performance outperforms other designs.**

***Index Terms*— Post-quantum cryptography, CRYSTALS-Kyber, lightweight hardware architecture, non-memory-based iterative NTT, compact SHA-3.**

I. Introduction

SECURITY, cost, and performance are the three most essential criteria when evaluating the PQC algorithms during the NIST Post-Quantum Cryptography Standardization Process [1]. Implementation cost is always prioritized for applications on constrained IoT devices. In the future, IoT devices are predicted to rapidly increase in number and play a critical role in core infrastructure. They are becoming more intelligent, smaller, and have a long life cycle. Hence, they must be secure even if a quantum computer exists [2]. It follows that constrained IoT devices need to be integrated with the PQC algorithms.

After the third round of evaluation, NIST recommended CRYSTALS-Kyber as the primary PQC key-establishment to be implemented for most applications. It has strong security and excellent performance and is suitable for constrained devices. Implementing the CRYSTALS-Kyber algorithm involves two primary operations: i) initializing symmetric primitives with hash functions from the FIPS-202 standards [3], and ii) performing addition, subtraction, and multiplication operations over the polynomial ring.

CRYSTALS-Kyber requires sampling a lot of random polynomials during key generation, encryption, and decryption processes. The Kyber team has selected the SHA3 standard with Keccak-f[1600] permutation for generating pseudo-random bits. This algorithm utilizes two hash functions, SHA3-256 and SHA3-512, and two extendable-output functions, SHAKE128 and SHAKE256. SHA3 is advantageous due to its hardware-friendliness and high bits output per round. However, all step mappings are performed in a 1600-bit state. This state is represented as a three-dimensional array containing 25 words, each of 64 bits in length. Hence, the truncated bits must be stored before rejective sampling or central binomial distribution (CBD). They can be temporarily stored in buffers [4], [5], [6] or fed into a parallel-in-serial-out unit [7]. Input data also requires hardware resources for large buffers. The Keccak team has proposed some low-resource implementations for SHA3 [8]. The state can be stored in memory, and step mappings are performed gradually. However, this method requires clock cycles to transfer data to or from the memory. Reference [9] has proposed dividing the round operation and performing it in parts. Their design has a relatively small hardware footprint. However, the computation time is significant and unsuitable for CRYSTALS-Kyber. Thus, standard SHA3 architecture is often implemented for CRYSTALS-Kyber [4], [6], [7], [10], [11], [12], [13]. It is easy to see that a compact hashing module is necessary for CRYSTALS-Kyber on constrained IoT devices.

Manuscript received 1 May 2024; revised 30 July 2024; accepted 9 August 2024. Date of publication 20 August 2024; date of current version 30 January 2025. This article was recommended by Associate Editor M. Ye. *(Corresponding author: Trong-Hung Nguyen.)*

Trong-Hung Nguyen, Cong-Kha Pham, and Trong-Thuc Hoang are with the Department of Computer and Network Engineering, The University of Electro-Communications (UEC), Tokyo 182-8585, Japan (e-mail: tronghung@vlsilab.ee.uec.ac.jp).

Duc-Thuan Dam is with the Department of Computer and Network Engineering, The University of Electro-Communications (UEC), Tokyo 182-8585, Japan, and also with the Faculty of Radio-Electronic Engineering, Le Quy Don Technical University (LQDTU), Hanoi 11917, Vietnam.

Phuc-Phan Duong is with the Department of Computer and Network Engineering, The University of Electro-Communications (UEC), Tokyo 182-8585, Japan, and also with the Faculty of Electronics and Telecommunications, Academy of Cryptography Techniques (ACT), Hanoi 12511, Vietnam.

Binh Kieu-Do-Nguyen is with the Department of Computer and Network Engineering, The University of Electro-Communications (UEC), Tokyo 182-8585, Japan, and also with the Faculty of Computer Science and Engineering, Ho Chi Minh City University of Technology (HCMUT), Ho Chi Minh City 740050, Vietnam.

Digital Object Identifier 10.1109/TCSI.2024.3443238

CRYSTALS-Kyber operates over the polynomial ring $R_{3329} = Z_q[X]/(X^n + 1)$ with module dimension $k = 2$, 3, and 4. Polynomial multiplication is the most complex and time-consuming part of CRYSTALS-Kyber implementation. The Kyber team has selected parameter sets $(n, q) = (256, 3329)$ that support fast polynomial multiplication based on the number theoretic transform (NTT). Consequently, the computational complexity is reduced from $O(n^2)$ to quasi-linear $O(n \log n)$. In CRYSTALS-Kyber, all calculations are performed in the NTT domain. The Cooley-Tukey (CT) algorithm [14] and Gentleman-Sand (GS) algorithm [15] efficiently transform polynomials between coefficient and point representations. In [16], Banerjee et al. proposed a unified butterfly unit architecture supporting CT and GS algorithms. Thus, the techniques for removing pre-process [17] and post-process [18] can be applied concurrently to eliminate costly bit-reversal operations.

The choice of NTT architecture concerns a trade-off between speed and hardware resources. Low latency is a critical criterion for large-scale cryptosystems requiring high performance. Pipelined NTT architectures such as single-path delay feedback (SDF) [19], [20] and multi-path delay commutator (MDC) [5], [20], [21], [22] are typically deployed. These methods use a chain of butterfly units to perform NTT and INTT quickly. Following each butterfly unit, a reordering unit rearranges the order of coefficients. These reordering units also act as temporary memory to hold coefficients after each stage. Therefore, pipelined NTT architecture does not require additional memory for coefficient storage and has simple memory access patterns. The main drawback of this architecture is that it requires a considerable number of butterfly and reordering units. In [22], a unified MDC architecture is proposed to reduce half of the reordering unit number. However, their hardware utilization is still high. Therefore, pipelined NTT is unsuitable for resource-constrained IoT devices.

Another widely used NTT architecture is the memory-based iterative NTT. This architecture requires an additional memory unit to store the iterative butterfly operation results at each NTT/INTT stage. Specifically, coefficients are read out from memory, perform the butterfly calculation, and then write the results back into memory. The number of butterfly units determines the execution speed of the NTT/INTT transform. Thus, the advantage of the iterative NTT lies in flexible choices for compact, high-performance, or balanced designs. However, its main drawbacks are the additional memory overhead and complex memory access patterns, particularly when using multiple butterfly units in parallel. In studies [16], [23], [24], [25], ping-pong memory techniques have been used to simplify memory access patterns. However, this technique doubles the memory cost. Therefore, many studies have proposed non-conflict memory access patterns on a shared memory unit for the iterative NTT architecture [26], [27], [28], [29], [30], [31]. Note that the point-wise multiplication (PWM) of CRYSTALS-Kyber is performed between two degree-1 polynomials. It follows that the above techniques are inefficient when directly applied to CRYSTALS-Kyber.

A. Related Works

Xing and Li [4] proposed a 2×1 NTT configuration for computing NTT and PWM in CRYSTALS-Kyber. Their hash module and NTT share the RAM bank to save memory resources. However, this design utilizes considerable FIFOs for data flow control within the hash module and NTT computation. In [11], Bisheh-Niasar et al. designed a 2×2 NTT configuration and a set of customized instruction codes to control data flow. However, their memory-addressing strategy is inefficient, leading to slow execution times. The work of [7] implemented k pairs of optimized 2×1 NTT configurations from [4], with $k = 2$, 3, and 4 corresponding to Kyber512, Kyber768, and Kyber1024 variants, respectively. Hence, their design achieved a balanced efficiency regarding hardware footprint and execution time across different security levels. In [13], Guo and Li proposed a conflict-free memory access pattern for mixed radix-2/4 NTT configuration. Their design achieved high bandwidth performance over hardware. In [32], the split-radix NTT configuration was adopted for CRYSTALS-Kyber. The authors also proposed a conflict-free memory mapping scheme for executing NTT and PWM. Another split-radix NTT design is proposed for high-speed CRYSTALS-Kyber implementation [33]. The results indicated the effectiveness of split-radix NTT compared to radix-2 NTT.

After surveying existing designs in the literature, we found that CRYSTALS-Kyber lacks sufficient study to be implemented in constrained IoT devices. Studies [4], [32] have implemented iterative NTT architectures with minimal butterfly configuration. However, iterative NTT methods require additional memory usage to store polynomial coefficients at each NTT stage. Additionally, the hashing module receives less attention than the NTT module. To our knowledge, existing designs all adopt standard hashing function architectures with significant hardware costs for FIFOs. Therefore, in-depth research into the core operations of CRYSTALS-Kyber is necessary for hardware resource optimization.

B. Our Contributions

This paper proposes an efficient hardware implementation of the lightweight CRYSTALS-Kyber PQC algorithm. Our contributions are summarized as follows:

1) We propose a compact SHA3 hash function architecture dedicated to CRYSTALS-Kyber. A buffer comprising 25 64-bit registers is designed for the 1600-bit state of the Keccak-f[1600] permutation. This buffer is controlled to execute the entire hashing process, including i) receiving the padded message, ii) performing 24 rounds of step mappings, and iii) outputting the pseudo-random bits. Hence, this architecture only consumes resources for the 1600-bit state, the combinational logic of $\theta - \rho - \pi - \chi - \iota$ functions in step mappings, and a minimal amount for padding. The FPGA implementation results show that the proposed SHA3 module consumes only 2828 LUTs, 1733 FFs, and 745 SLICEs, reducing hardware costs by 29% to 60% compared to previous studies.
2) A non-memory-based iterative NTT (NMI-NTT) architecture is proposed for the first time in the literature.

We design a dual-butterfly unit to compute NTT/INTT and PWM operations. Then, we design a configurable reordering unit (RU) to rearrange the order of coefficients at all NTT/INTT stages of each butterfly unit. During each computational stage, our NMI-NTT also serves as a temporary memory for storing all coefficients of one polynomial. The proposed NMI-NTT eliminates the need for additional memory usage in the iterative NTT design, simplifying the control logic. NMI-NTT design consumes 448 CCs for NTT/INTT and 256 CCs for PWM. With 1005 LUTs, 599 FFs, 319 SLICEs, 0 BRAMs, and 2 DSPs, our hardware footprint is the smallest, and the ATP significantly improved from 19% to 88% compared to other iterative NTT designs.

3) Our hash module has relatively slow execution times, especially for hashing long messages (ciphertext ct, public key pk) or repeatedly executing the function f-maps strings of SHAKE128 and SHAKE256 functions. To address this issue, we propose a tightly coupled data flow for the hash module, sampling, and NTT. Generating pseudo-random bit streams and sampling polynomials is parallelized with NTT and PWM computations. As a result, the latency for generating the randomness polynomials is absorbed mainly by the operations of the NMI-NTT module.
4) We adopt the above novel designs for our lightweight CRYSTALS-Kyber on Artix-7 FPGA (as recommended by NIST). As a result, our CRYSTALS-Kyber implementation reports the lowest hardware cost compared to the state-of-the-art studies, with 5487 LUTs, 3426 FFs, 1548 SLICEs, 3.5 BRAM, and 2 DSPs. Our ATP outperforms other iterative NTT-based implementations, achieving a maximum improvement of up to 80%. It is simple to see that our CRYSTALS-Kyber design is suitable for implementation on constrained IoT devices.

II. Preliminary

A. CRYSTALS-Kyber

CRYSTALS-Kyber is a cryptographic primitive in the Cryptographic Suite for Algebraic Lattices (CRYSTALS). It is an IND-CCA2-secure key encapsulation mechanism (KEM) based on the hardness of the module-learning with error (M-LWE) problem [34]. CRYSTALS-Kyber's algebraic operations are on the polynomial ring $R_q = \mathbb{Z}_q[x]/(x^n + 1)$. The latest version of CRYSTALS-Kyber has selected the parameter set $(n, q) = (256,3329)$ to reduce the size of the public key and cipher text. There are three security levels of CRYSTALS-Kyber: Kyber512, Kyber768, and Kyber1024, corresponding to NIST Security levels 1, 3, and 5, respectively. Its security level is defined by the module dimension-k (polynomial matrix and polynomial vector). The primary operations of CRYSTALS-Kyber are listed as follows, where sk stands for the secret key, pk for the public key, ct for ciphertext, and m for the message:

- **Key generation**: Initialize a pair seed (ρ, σ); use seed ρ to generate polynomial vectors $\mathbf{s}$ and $\mathbf{e}$ from CBD sampling and polynomial matrix $\mathbf{A}$ from rejective sampling. The secret key is $sk = \mathbf{s}$, and the public key is $pk = (\mathbf{t}, \rho)$, where $\mathbf{t} = \mathbf{As} + \mathbf{e}$.
- **Encryption**: Using seed σ to generate polynomial vector $\mathbf{r}$,$\mathbf{e}_1$, and polynomial e_2 from CBD sampling. Generating $\mathbf{A}$ and the cipher text is $ct = (\mathbf{u}, v)$ where $\mathbf{u} = \mathbf{A}^T\mathbf{r} + \mathbf{e}_1$, $v = \mathbf{t}^T\mathbf{r} + e_2 + m$.
- **Decryption**: Computing $m = v - \mathbf{s}^T\mathbf{u}$.

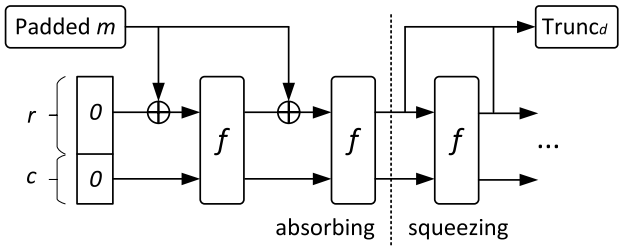

Fig. 1. The sponge construction [3].

TABLE I
THE SPECIFICATION OF SHA3 STANDARD IN CRYSTALS-KYBER

Symmetric primitives	SHA3 functions	r	c	Padding rules
H	SHA3-256	1088	512	$m\|\|01\|\|1\|\|0^l\|\|1$
G	SHA3-512	576	1024	$m\|\|01\|\|1\|\|0^l\|\|1$
XOF	SHAKE128	1344	256	$m\|\|j\|\|i\|\|1111\|\|1\|\|0^l\|\|1$
PRF, KDF	SHAKE256	1088	512	$m\|\|(N)\|\|1111\|\|1\|\|0^l\|\|1$

B. SHA3 Standard

SHA-3 is the latest secure hash algorithm released by NIST in 2015, based on the Keccak-f[1600] permutation [3]. The SHA-3 functions share a sponge construction consisting of three components: a function f maps strings, a rate parameter r, and a padding rule, denoted by pad, as depicted in Fig 1. The length of the function f is referred to as b, and b = 1600. The rate r is a positive integer less than b, and the capacity $c = b - r$. The padding rule involves creating a string of length that is a positive multiple of r from the input data m. Implementing a Keccak permutation requires performing 24 rounds of five-step mappings. CRYSTALS-Kyber utilizes the SHA3-256 and SHA3-512 hash functions and SHAKE128 and SHAKE256 extendable-output functions. Therefore, the padding rule differs when initializing the symmetric primitives of CRYSTALS-Kyber. Tab I illustrates the specification of the SHA3 functions in CRYSTALS-Kyber.

C. Number Theoretic Transform

The Number Theoretic Transform (NTT) is a fast computing method for polynomial multiplication. It is a generalization of the fast Fourier transform in the finite field. In [35], Lyubashevsky et al. proposed the negative-wrapped convolution (NWC) to avoid zero-padding for NTT/INTT transformations. However, the NTT in CRYSTALS-Kyber cannot directly adopt the NMC method. Thus, the NTT of polynomial f is expressed as NWC-based NTT of the two parity element polynomials of f. The NTT(f) is defined as follows:

$$NTT(f) = \hat{f} = \hat{f}_{2i} + \hat{f}_{2i+1}X, \tag{1}$$

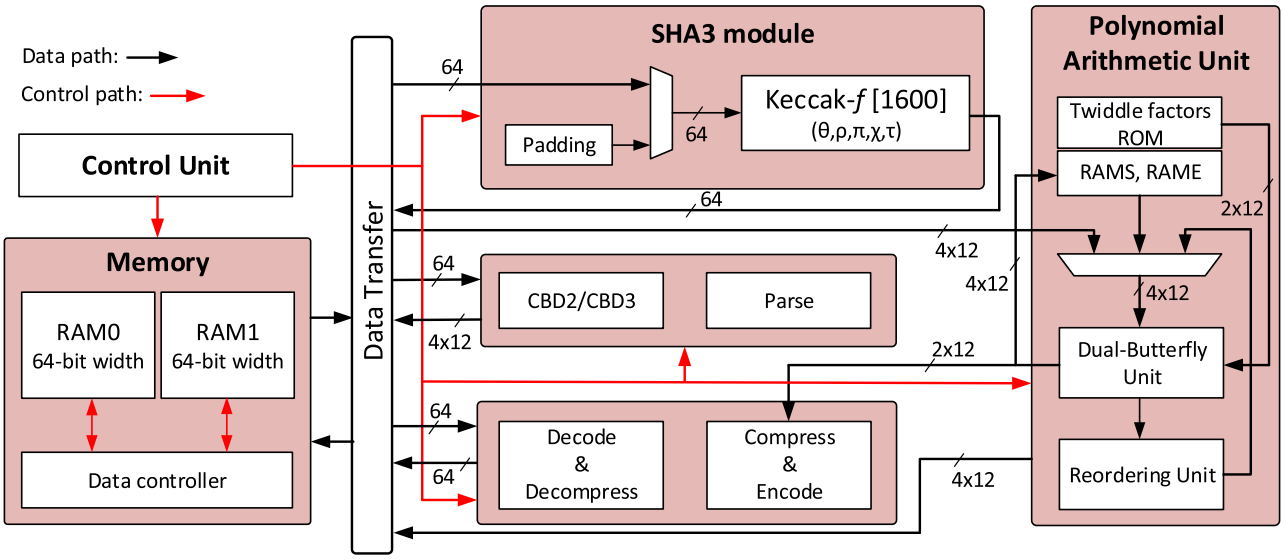

Fig. 2. The overall architecture of the CRYSTALS-Kyber. The red lines show control signals, and the black lines show data movement.

where $\hat{f}_{2i}$ and $\hat{f}_{2i+1}$ are defined as,

$$\hat{f}_{2i} = \sum_{j=0}^{127} f_{2j}\zeta^{(2br_7(i)+1)j} \tag{2}$$

$$\hat{f}_{2i+1} = \sum_{j=0}^{127} f_{2j+1}\zeta^{(2br_7(i)+1)j} \tag{3}$$

with ζ is the 256-th root of unity in $\mathbb{Z}_q$ and $i = 0,\ldots,127$ is a 7-bit unsigned integer. The multiplication of two polynomials f, g over the ring $R_q = \mathbb{Z}_q[x]/(x^n + 1)$ is performed as $h = f \times g = INTT(\hat{f} \circ \hat{g})$, where $\hat{h} = \hat{f} \circ \hat{g}$ is defined as,

$$\hat{h} = \hat{h}_{2i} + \hat{h}_{2i+1}X = (\hat{f}_{2i} + \hat{f}_{2i+1}X)(\hat{g}_{2i} + \hat{g}_{2i+1}X) \tag{4}$$

Thus, the point-wise multiplication (PWM) in CRYSTALS-Kyber is performed by five modular multiplications on $\mathbb{Z}_q$ as follows:

$$\hat{h}_{2i} = \hat{f}_{2i}\hat{g}_{2i} + \hat{f}_{2i+1}\hat{g}_{2i+1} \cdot \zeta^{2br(i)+1} \tag{5}$$

$$\hat{h}_{2i+1} = \hat{f}_{2i}\hat{g}_{2i+1} + \hat{f}_{2i+1}\hat{g}_{2i} \tag{6}$$

III. Lightweight CRYSTALS-Kyber Architecture

Fig. 2 shows the overall architecture of the proposed lightweight CRYSTALS-Kyber. The Keccak-based SHA3 module and the Polynomial Arithmetic Unit are two primary parts of the design. The design details will be described in the following subsections. In addition, our SHA3 module is a trade-off between hardware resources and speed. Hence, sampling and performing polynomial arithmetic must be coupled with memory organization for timing optimization. We will discuss how we efficiently manage the memory and propose the optimized execution data flow.

A. Memory Arrangement

In [4], [5], and [7], FIFOs are required to hold data results and samples. This technique simplifies memory access patterns and control signals in CRYSTALS-Kyber. However, it wastes memory bandwidth and an inefficient hardware footprint. The data length in CRYSTALS-Kyber is multiples of 64 bits. The SHA3 module also operates on the 64-bit data flow. In order to reduce memory usage, we designed a 64-bit width for the data flow in CRYSTALS-Kyber. Then, all data are stored in the main memory, consisting of two memory banks: RAM0 and RAM1, as in Fig 3.

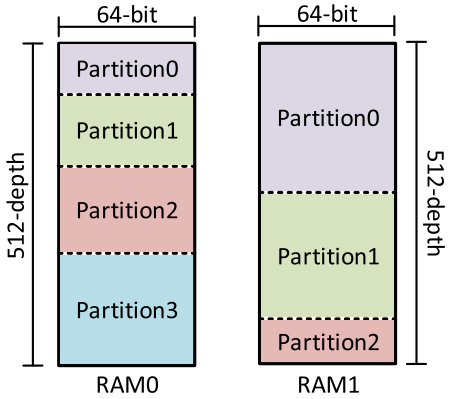

Fig. 3. The main memory organization.

RAM0 stores input and output data for the SHA3 module, excluding the public key pk. RAM0 contains four data partitions: 0, 1, 2, and 3. Partition 0 stores the seeds $(z, \rho, \sigma, \bar{K}, r, \bar{K}', r')$ for initializing the symmetric primitives of rejective sampling and CBD. The hash values of functions $H(pk)$, $H(m)$, $H(ct)$, and KDF are in Partition 1. Partition 2 contains pseudo-random numbers generated by the instantiation XOF and PRF. Partition 3 is for storing the ciphertext ct. Besides, RAM1 involves three data partitions: 0, 1, and 2. The secret key sk and the public key pk are stored in Partitions 0 and 1. Partition 2 is a buffer to store one polynomial during sampling or $Decompress(u, du)$, $Decompress(v, dv)$ functions. The size of RAM0 and RAM1 is 64×512, sufficient to store data in all three security levels of CRYSTALS-Kyber.

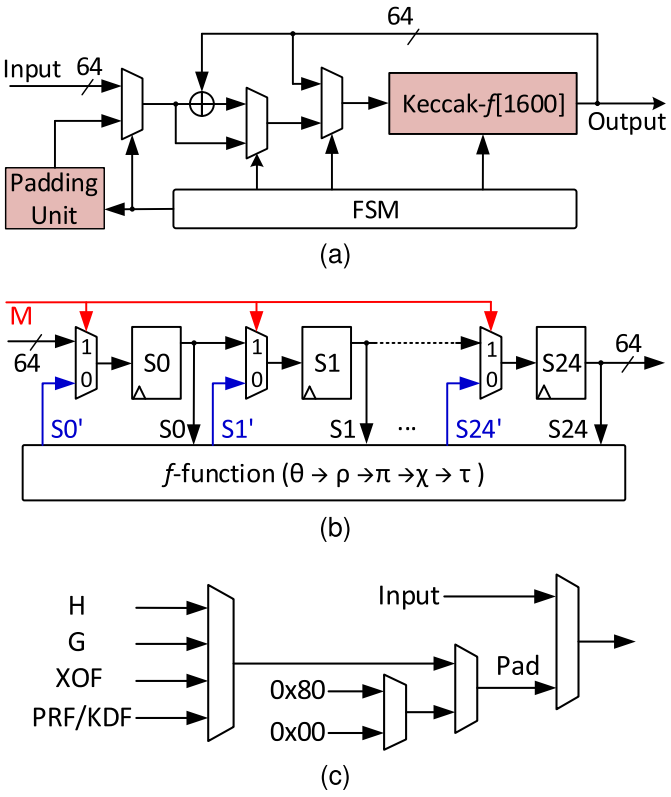

Fig. 4. The architecture of the proposed SHA3 module (a) The sponge construction (b) The In-Hash-Out Keccak permutation structure (c) Padding scheme.

Partitioning memory into multiple data regions requires complex memory access patterns. However, as a bonus of our compact SHA3, reading and writing data are performed sequentially. Furthermore, we optimize data flow for CRYSTALS-Kyber NTT operations to avoid conflict memory access. The following sections will analyze the details of these optimizations.

B. Compact SHA3 Hashing Module

In hardware implementation, a 1600-bit state array is represented by 25 64-bit width registers. The basic Keccak permutation executes 24 rounds of five-step mappings, equivalent to 24 clock cycles (CCs). Five-step mappings update the state within one CC. Therefore, the input data is padded and stored in a buffer beforehand. In CRYSTALS-Kyber, the SHA3-256 hash function is utilized for hashing the ciphertext (ct). Thus, the hash module requires a buffer size of up to 64×21 bits, nearly equivalent to the state size. Similarly, the output data is temporarily stored in a buffer for post-processing.

Fig 4a illustrates the architecture of our SHA3 module. The sponge construction is designed with a standard 64-bit width data flow. It includes a Keccak-$f[1600]$ permutation, a padding unit, a 64-bit logical XOR, and a finite state machine (FSM). The detailed structure of the proposed Keccak-$f[1600]$ permutation is depicted in Fig 4b. We propose a method to load data directly into the state. The input of 25 state registers ($S0, S1, \ldots, S24$) is controlled to perform the f-function or act as a shift register. When $M = 1$, the state is fed with data from the input or reads data out for the output. When $M = 0$, the f-function is executed to update the state. Hence, the Keccak-$f[1600]$ permutation operates as an in-hash-out data flow. In CRYSTALS-Kyber, each hash function requires a single padded word, as shown in Tab I. Our padding scheme is depicted in Fig 4c. The FSM selects standard padded words (0×80, 0×00) in the loading mode. Our SHA3 module eliminates using large buffers for input and output data.

The limitation of our compact SHA3 module is its hashing time. One full 24-round Keccak-$f[1600]$ permutation consumes 74 CCs, corresponding to 50 CCs for loading data into and out of the state and 24 CCs for hashing. In the implementation of CRYSTALS-Kyber, the hashing time significantly affects the random polynomial sampling process. Specifically, the public matrix $\hat{\mathbf{A}}/\hat{\mathbf{A}}^T$ is sampled through the random source of the XOF from the SHAKE128 function. The coefficient is generated by rejection-based sampling Parse function with an acceptance rate of 81.27%. Our Parse unit is designed to generate a pair of coefficients. Therefore, the XOF function outputs 1344 pseudo-random bits in one 24-round permutation, giving a probability of $(1344/24) \times 81.27\% \times 81.27\% = 36$ valid samples. Thus, at least four XOFs are required to generate one polynomial of the matrix $\hat{\mathbf{A}}/\hat{\mathbf{A}}^T$.

The process of generating $\hat{A}_{ij}$ has three stages: 1) SHA3 module reads seed-ρ from Partition 0 of RAM0 and initializes the XOF function, 2) Hashing four times, and the hash values of each time are sequentially stored in Partition 2 of RAM0, 3) When the first 64-bit hash value is stored, the Parse module starts to read out, generates valid coefficients, and stores them in Partition 2 of RAM1. Our SHA3 module consumes 266 CCs for one polynomial of matrix $\hat{\mathbf{A}}/\hat{\mathbf{A}}^T$. In the case of generating noise polynomials ($\mathbf{s}, \mathbf{e}, \mathbf{r}, \mathbf{e1}, e2$), the $CBD_3(\eta_1 = 3)$ unit requires two hash operations, while the $CBD_2(\eta_1/\eta_2 = 2)$ unit requires just one. Our SHAKE256 function takes 49 CCs to generate 1088 pseudo-random bits. The CBD_3 unit, on the other hand, requires $1088/24 = 46$ CCs to handle all 1088 pseudo-random bits. Hence, the CBD_3 sampling rate is faster than the hashing rate of the SHAKE256 function. To ensure continuous sampling, our CBD_3 unit initiates sampling after 16 CCs of the first hash result. As a result, our SHA3 module requires 138 CCs for CBD_3 and 122 CCs for CBD_2, with 64 CCs for sampling and the rest for hash operations.

C. Polynomial Arithmetic

The polynomial arithmetic unit (PAU) is designed to perform operations over the ring R_q for all three CRYSTALS-Kyber security levels. This unit can execute NTT and INTT operations. It also performs PWM and accumulation for computing matrix-vector and vector-vector polynomial multiplications. PAU module consists of the dual-butterfly unit, ROM, RAM, and reordering unit (RU). The butterfly unit handles NTT, INTT, PWM, and accumulation operations. The ROM stores the twiddle factors. The RAM includes two banks: i) RAMS stores noise polynomial vectors in the NTT domain ($\hat{\mathbf{s}}/\hat{\mathbf{r}}/\hat{\mathbf{u}}$) used for PWM operations; ii) RAME stores the results of PWM and accumulation operations. RAMS has

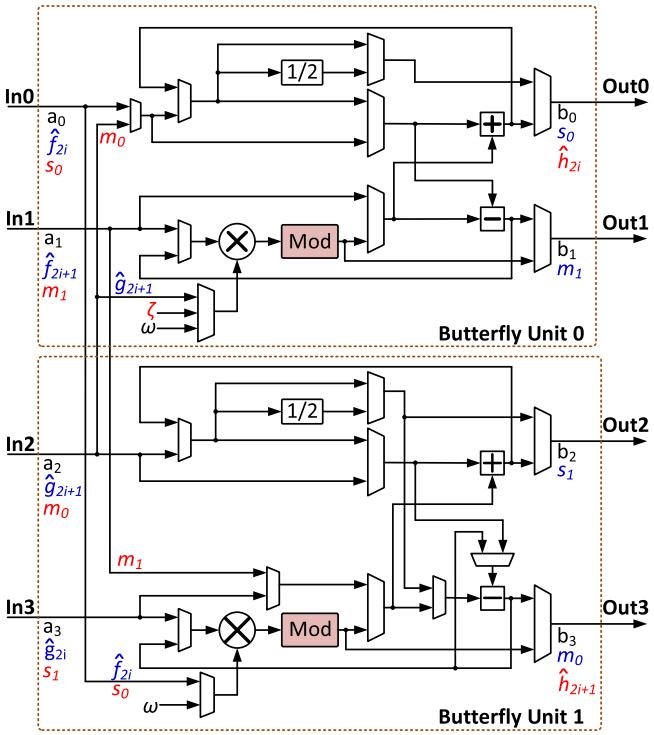

Fig. 5. The proposed double-butterfly unit for CRYSTALS-Kyber. It can be configured to perform NTT/INTT (black-colored coefficients), first-stage PWM (blue-colored coefficients), and second-stage PWM (red-colored coefficients).

a data width of 4×12 bits (4 coefficients) and a depth of 6×64 (4 polynomials of $\hat{\mathbf{s}}/\hat{\mathbf{r}}/\hat{\mathbf{u}}$ in Kyber1024 and one result of the first step in PWM operations). RAME has a data width of 4×12 bits (4 coefficients) and a depth of 1×128 (one result of PWM and accumulation operations). The RU unit performs two tasks, including reordering and holding coefficients of the polynomial during the NTT/INTT process.

1) Dual-Butterfly Unit: Using the NWC-NTT method [35] directly with the latest parameter set of the CRYSTALS-Kyber polynomial is impossible. Therefore, a 256-term polynomial is divided into two 128-term polynomials by parity. The NWC-NTT is applied independently to each parity polynomial. We design a dual-butterfly unit consisting of two parallel radix-2 butterflies, as shown in Fig 5. The CT-GS unified butterfly configuration eliminates the pre-process, post-process, and bit-reversal steps. For the multiplication by n^{-1}, we use a division by 2 at each layer of the INTT. The dual-butterfly unit can perform parallel butterfly operations for even and odd coefficient polynomials.

All butterfly operations involve modular arithmetic. The modular multiplication $a \times b = c\,(mod\,q)$ is complex and resource-intensive. Study [36] has summarized modular reduction methods and proposed a dedicated digital signal processing (DSP) for CRYSTALS-Kyber. However, in this design, we leverage the available DSP for integer multiplication and propose a modular reduction unit (Mod). The polynomial coefficient has a width of 12 bits and ranges from $[0:3329)$. The product of two coefficients has a width of 24 bits, with the most considerable value being 3328×3328. Therefore, we utilize the pre-computed lookup table (LUT) to store computations of $c0 = c[17:12]\times2^{12}\,(mod\,3329)$ and $c1 = c[23:18]\times2^{18}\,(mod\,3329)$. The LUT for $c1$ contains 42 values, while the LUT for $c0$ contains 64. Finally, the modular reduction is performed using modular addition $c[11:0]+c0+c1\,(mod\,3329)$.

Our dual-butterfly unit can also perform PWM. In [4], the authors adopted the Karatsuba algorithm to reduce the number of multiplications in equations (5) and (7) to just four. Specifically, equations (5) and (7) are transformed as follows:

$$\hat{h}_{2i} = \hat{f}_{2i}\cdot\hat{g}_{2i} + \hat{f}_{2i+1}\cdot\hat{g}_{2i+1}\cdot\zeta^{2br(i)+1} \tag{7}$$

$$\hat{h}_{2i+1} = (\hat{f}_{2i}+\hat{f}_{2i+1})\cdot(\hat{g}_{2i}+\hat{g}_{2i+1}) - \hat{f}_{2i}\cdot\hat{g}_{2i} - \hat{f}_{2i+1}\cdot\hat{g}_{2i+1} \tag{8}$$

Performing PWM involves two stages. In the first stage, operations $s_0 = \hat{f}_{2i}+\hat{f}_{2i+1}$, $s_1 = \hat{g}_{2i}+\hat{g}_{2i+1}$, $m_0 = \hat{f}_{2i}\times\hat{g}_{2i}$, and $m_1 = \hat{f}_{2i+1}\times\hat{g}_{2i+1}$ are calculated. The (s_0, s_1, m_0, m_1) results are stored in RAMS. In the second stage, calculations $\hat{f}_{2i} = m_0 + m_3$ and $\hat{f}_{2i+1} = m_2 - s_2$ are carried out, where $s_2 = m_0 + m_1$, $m_2 = s_0\times s_1$, and $m_3 = m_1\times\zeta^{2br(i)+1}$. The results of $\hat{f}_{2i}$ and $\hat{f}_{2i+1}$ are stored in RAME. The details of the computations in PWM are illustrated in Fig 5. Three modes, NTT, INTT, and PWM, are selected by controlling the multiplexers. A single PWM multiplication in our dual-butterfly unit consumes 256 CCs.

2) Non-Memory-Based Iterative NTT: As mentioned in Section I, additional memory and memory access patterns are two critical parts of the iterative NTT hardware design. In the case of a highly parallelized NTT architecture, these problems become even more challenging. Implementing the NTT requires changing the order of coefficients after each stage. Therefore, the data address pattern is changed. The difference in memory access patterns complicates NTT design. Besides, a large memory is needed to store coefficients at multiple stages. Instead of using the read-write-on memory technique, we propose performing NTT in place.

Fig 6a illustrates the proposed non-memory-based iterative NTT (NMI-NTT) architecture for CRYSTALS-Kyber. It has a butterfly unit (BU) and a reordering unit (RU). During the first stage of NTT/INTT execution, the input multiplexer receives the polynomial coefficients. Next, the BU performs the CT or GS butterfly operation. Like the MDC architecture [22], the RU rearranges the result coefficients through two delay units (D0) and multiplexers. In particular, D0 can be configured to generate the delay for all stages. Next, an additional delay unit (D1) holds the reordered coefficients to be ready at the input multiplexer for the subsequent stage. After the final NTT/INTT stage, the NMI-NTT reads the results from (Out0, Out1) and then executes the next polynomial. Our NMI-NTT functions like a memory array, holding the result coefficients within one NTT/NTT stage's execution time. Hence, this memory array is minimal in size for storing one polynomial.

To explain how our NMI-NTT works, we utilize a 16-point NTT as an example. Fig 6b illustrates the design of the delay unit D0. Besides, the execution data flow of the forward-NTT is depicted in Fig 6c. Considering the 16-point forward NTT, the offset of input coefficients at each stage are 8, 4, 2, 1. Assuming the coefficients are initially generated

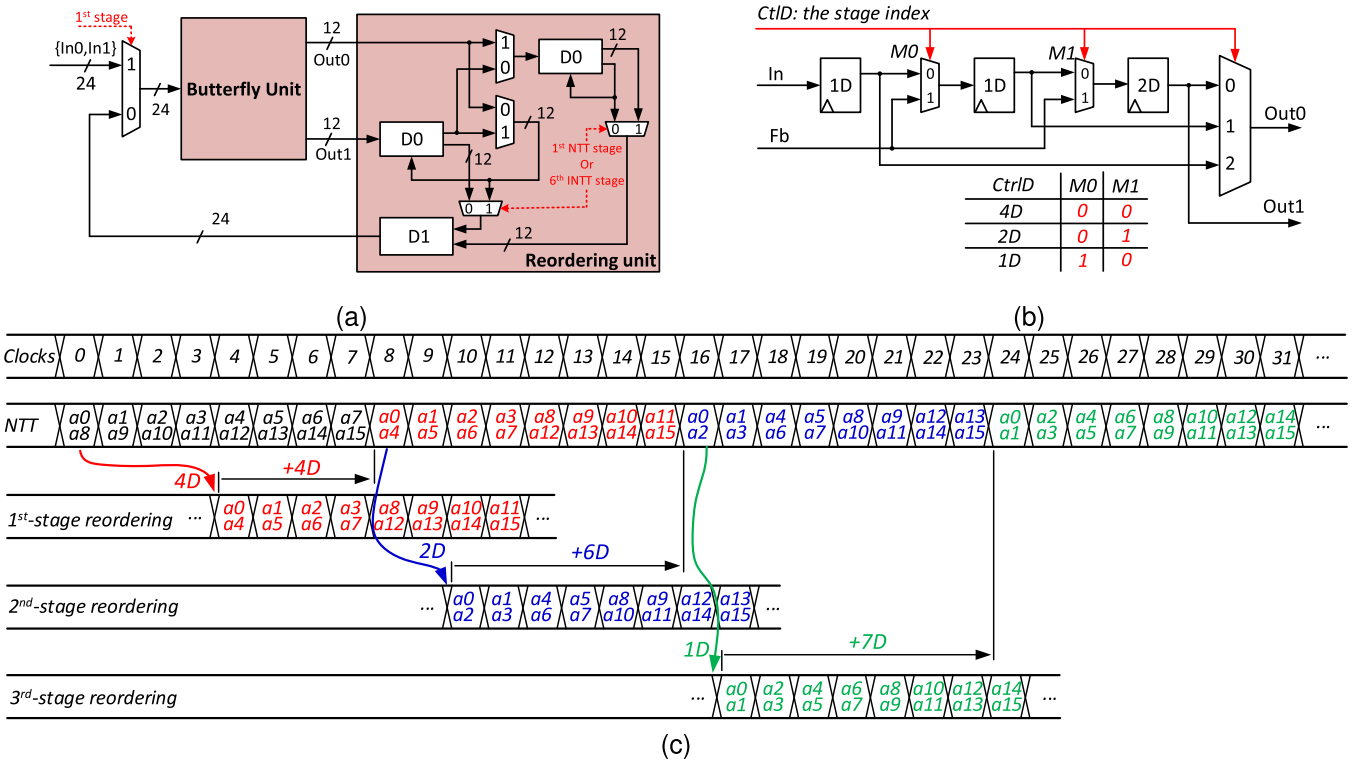

Fig. 6. The proposed NMI-NTT architecture. (a) The top level of the NMI-NTT design. (b) The segmented delay unit D0 for 16-point NTT. (c) The execution data flow of 16-point forward-NTT.

with an offset of 8. Hence, the output coefficients through D0 must be delayed (D) by 4D, 2D, and 1D at the corresponding stages.

We propose a segmented delay unit D0 to save hardware utilization, as shown in Fig 6b. Specifically, a D0 with a 4D delay is divided into two 1D delays and one 2D delay. The multiplexers M0 and M1 are controlled to generate the delay at each NTT stage. The control signal $CtlD$ is shared for the switch commuter and D0. The logic table of M0 and M1 is also depicted in Fig 6b. For example, in the first NTT stage, M0 = M1 = 0, the delay is 1D + 1D + 2D = 4D. When M0 = 0 and M1 = 1, the delay is 1D + 1D = 2D, corresponding to the second NTT stage. Finally, when M0 = 1 and M1 = 0, the coefficient is delayed by 1D before appearing at $Out0$. As a result, the order of coefficients is arranged by a minimal hardware unit.

Then, we discuss how coefficients are stored during the NTT process. First, we consider the number of coefficients in one polynomial. A 16-point NTT corresponds to 8 coefficient pairs. The two delay units D0 can hold up to four coefficient pairs. Therefore, an additional storage unit D1 for the remaining coefficients is required. Note that pipelined registers in the hardware design also serve as memory units. As a bonus regarding the area, the depth of D1 is $(4 - x)$D, with x being the number of pipelined registers in the BU architecture.

However, D0 can only hold four coefficient pairs at the first reordering stage, as illustrated in Fig 6c. In the second stage, coefficients are only held for 2 CCs. Therefore, input data conflict occurs at butterfly operations (a_{10}, a_{14}) and (a_{11}, a_{15}). Similarly, operations(a_9, a_{11}), (a_{12}, a_{14}) and (a_{13}, a_{15}) are conflicted in the third stage. To address this issue, we control the coefficient flow to feed back into D0. Specifically, in cases where D0 $<$ 4D, the coefficients are fed back into the unused delay units in D0 after passing through the switch. For example, at the second reordering stage where D0 = 2D, the feedback coefficient passes through M1 and is delayed by 2D before appearing at $Out1$. Besides, the feedback coefficient is delayed by 3D through M0 and M1 at the third reordering stage. Therefore, coefficients are always held for 4D at D0 during the NTT process.

Our NMI-NTT architecture only requires controlling the $CtlD$ signal to execute either forward or inverse NTT. In the case of CRYSTALS-Kyber, this architecture consumes 448 CCs for one NTT transformation.

D. Optimizing Execution Data Flow

The above III-B subsection analysis shows that our SHA3 module requires significant computation time. Besides, all data is stored in two main memories, RAM0 and RAM1. Therefore, the CRYSTALS-Kyber functions are limited in execution time and parallelism. Most of the execution time in CRYSTALS-Kyber is taken by performing NTT/INTT and PWM. To reduce CCs, the execution data flow needs to be tightly scheduled. Memory-based operations such as Encode/Decode and Compress/Decompress or sampling should be hidden by overlapping with matrix-vector and vector-vector polynomial multiplications. Furthermore, operations between modules require tight coordination to avoid

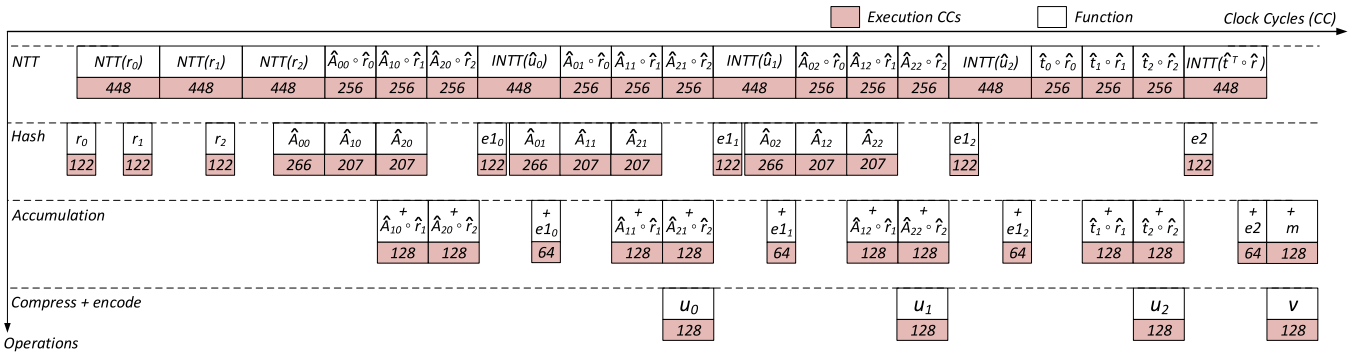

Fig. 7. The execution order and clock cycles for primary operations in Kyber.CPA.Enc of Kyber768.

memory conflicts. To allow conflict-free memory, memory-based operations must be executed sequentially.

In Fig 7, we can see how the execution data flow of Kyber.CPA.Enc is optimized for Kyber768. The primary data flow consists of various functions such as polynomial generation, NTT/INTT, PWM, accumulation, and compression. The number of consumed CCs is shown below each function. At first, the noise polynomial r_0 is generated by CBD2 in 122 CCs. The CBD2 function can generate four coefficients per CC. Hence, the $NTT(r_0)$ begins when the first two pairs of coefficients are generated. For generating subsequent polynomials, two conditions need to be fulfilled: 1) only one polynomial is generated at a time, and 2) a new polynomial is generated only when the coefficients of the old polynomial start to be read out. The reason is that each polynomial's seed, pseudo-random bit stream, and coefficients are stored at the same address in RAM0 and RAM1. Hence, the noise polynomials (r_0, r_1, r_2) are generated before the NTT execution. In the case of the public matrix $\hat{\mathbf{A}}$, creating an element polynomial $\hat{A}_{ij}$ consumes 266 CCs, whereas one PWM consumes 256 CCs. However, for subsequent $\hat{A}_{ij}$ polynomials, the generation time only consumes 207 CCs. Therefore, the two conditions mentioned above are still ensured.

The accumulation operation is performed in PWM or INTT execution, and the result is stored in RAME. The accumulation $u_0 = INTT(\hat{u}_0) + e1_0$ is executed while generating $\hat{A}_{01}$. Therefore, to avoid memory conflicts with $\hat{A}_{01}$ in RAM1, $e1_0$ is generated and directly stored in RAME. Similarly, performing $u_1 = INTT(\hat{u}_1)+e1_1$, $u_2 = INTT(\hat{u}_2)+e1_2$ and $v = INTT(\hat{\mathbf{t}}^T \circ \hat{\mathbf{r}}) + e2$ are handled in this manner. Besides, the cipher text u_0 is compressed and stored in RAM0. To avoid conflicts with the generation of $\hat{A}_{j1}$, the $Compress(u_0, du)$ is carried out during the execution of $\hat{A}_{21} \circ \hat{r}_2$. The same method is applied to $Compress(u_1, du)$ and $Compress(u_2, du)$.

The encryption of message m, $v = INTT(\hat{\mathbf{t}}^T \circ \hat{\mathbf{r}}) + e2 + Decompress(m, 1)$, and $Compress(v, dv)$ are implemented in parallel. The compression consumes 128 CCs for each element ciphertext. It can be observed that NTT operations predominantly absorb the execution time of sampling, encryption, and compression in Kyber.CPA.Enc. As a result, the total CCs required for Kyber.CPA.Enc execution is reduced. These optimizations are similarly applied to Kyber.CPA.KeyGen and Kyber.CPA.Dec.

In addition, to configure different security levels of CRYSTALS-Kyber, we rely on the number of NTT and PWM computations. To determine the number of processed NTT and PWM operations, we use two small counters: a 3-bit counter for NTT and a 5-bit counter for PWM. These counters generate the finite-state machine's state control signals. As a result, our design can support three security levels without requiring hardware overhead.

IV. Implementation Results and Comparisons

A. Implementation Results

The proposed hardware architecture for the lightweight CRYSTALS-Kyber is implemented on the Xilinx Artix-7 AC701 Platform with Xilinx Vivado 2021.2. All implemented results have been obtained after synthesis, place-and-route with default settings. Our design supports three CRYSTALS-Kyber security levels. The proposed architecture consumes 5.5K LUTs, 3.4K FFs, 1.5K SLICEs, 3.5BRAMs, and 2 DSPs with an operating frequency of 185 MHz. Tab II shows the hardware utilization breakdown of the main modules in this design. SHA3 is the most hardware-intensive module, taking up half of the slices. Besides, the polynomial arithmetic unit takes for about one-fourth of the area. The remaining resources are used to sample Parse/CBD, perform Compress/Decompress and Encode/Decode functions, and memory control data. Regarding time performance, our design completes key generation, encapsulation, and decapsulation phases in Kyber.CCAKEM within 17.8/24.3/32.9 μs for Kyber512, 30.3/38.4/49.7 μs for Kyber768, and 45.9/54.6/69.7 μs for Kyber1024.

B. Comparison With Other Works

In Tabs III, IV, and V, we report the implementation results of our optimization and compare them with previous work. To ensure fair comparisons, we report the Area-Time trade-off as a product of Total Cycle Count (Keygen + Encapsulation + Decapsulation) × Equivalent Number of Slices (ENS). ENS is the sum of reported SLICEs and the converted slices from DSP and BRAM. We use the most common conversion ratio of 1 DSP = 100 SLICEs and 1 BRAM = 200 SLICEs. In cases where SLICEs are not reported, we calculate from LUTs and

TABLE II
THE HARDWARE RESOURCE UTILIZATION OF THE PROPOSED LIGHTWEIGHT CRYSTALS-KYBER

Module	LUTs	FFs	SLICEs	BRAMs	DSPs
SHA3	2828	1733	745	0	0
Polynomial arithmetic	1416	1074	431	1.5	2
⌊ NTT/PWM	1005	599	319	0	2
⌊ BU	400	260	130	0	2
⌊ Mod (×2)	136	52	38	0	0
⌊ RU (×2)	352	144	128	0	0
$CBD_{2/3}$,Parse	116	141	62	0	0
Encode, Compress	198	212	101	0	0
Decode, Decompress	112	156	82	0	0
Memory control	397	58	89	2	0
Total	5487	3426	1548	3.5	2

TABLE III
COMPARISON OF OUR SHA3 CORE WITH PREVIOUS WORK

Work	LUTs	FFs	SLICEs	Area Optimization
[4]	4014	1980	1251	1.68 (+40%)
[37]	4614	1771	1407	1.89 (+47%)
[6]	6841	4279	1873	2.51 (+60%)
[13]	3969	1690	1045	1.40 (+29%)
This work	**2828**	**1733**	**745**	**1.0 (0%)**

FFs, corresponding to 1 SLICE = 4×LUTs + 8×FFs for Artix-7 FPGA platform [22].

In this study, we prioritize area optimizations. Tab III reports the hardware resource results and comparisons of the most advanced SHA3 hashing modules for CRYSTALS-Kyber. Studies [4], [6], [13], [37] are all designed with a standard Keccak-f[1600] state unit and use FIFOs for data in/out. Besides, study [6] designs a sponge structure with a data buffer width of up to 1344 bits. Therefore, this design requires the most resources. The results show that our SHA3 significantly reduces area with area optimization of 29%, 40%, 47%, and 60% compared to [4], [6], [13], and [37]. However, this result is traded off with hashing time. To avoid impacting the overall clock cycles, generating polynomials has been hidden behind the operations of the polynomial arithmetic unit.

Tab IV presents the results of iterative NTT implementations for CRYSTALS-Kyber. All compared designs have been attempted to optimize the usage and access patterns for NTT memory. Conversely, the proposed NMI-NTT consumes fewer hardware resources by eliminating additional memory. Our NMI-NTT supports three modes: NTT/INTT/PWM with respective execution times of 448/448/256 CCs. Our hardware utilization is 1005 LUTs, 599 FFs, 319 SLICEs, 0 BRAMs, and 2 DSPs. Some studies do not report PWM execution time. Therefore, we report the Area-Time trade-off as the product of Total Cycle Count (NTT + INTT) × ENS. Study [38] proposes a single butterfly configuration for CRYSTALS-Kyber NTT. This design requires up to 2.5 BRAMs and numerous CCs to perform PWM. Our NMI-NTT performs better in ATP, with a 71% improvement.

Studies [4], [7] and ours select the dual-butterfly configuration. In [7], Dang et al. employ head/tail units to reorder coefficients in BU input and output. Thus, this design has better ATP compared with [4]. Our design has the smallest ENS and outperforms in terms of ATP efficiency by 38% and 68% compared to [4] and [7]. Studies [11], [13], [25], [39], [41] implemented NTT for multiple butterfly operations. References [11], [25], and [39] focus on memory optimization and have fewer BRAM usage. However, their designs perform relatively slow NTT/INTT/PWM. Study [13] proposes a memory bank for mix-radix2/4 NTT and performs NTT/INTT in 225 CCs. In return, 4 BRAMs are required. In study [41], Li et al. combined four butterfly units to form a bi-core and proposed an efficient parallel memory access pattern. Their NTT achieves better ATP hardware efficiency than other memory-based iterative NTT designs. Therefore, our NMI-NTT has better ATP with a significant improvement of 19%, 34%, 42%, 73%, and 88% compared with [11], [13], [25], [39], and [41].

In [32], a compact split-radix NTT is designed for CRYSTALS-Kyber. This architecture outperforms the radix-2 NTT and needs 3 BRAMs for memory footprint. Besides, [33] proposed a split-radix NTT with further optimizations for PWM operation. Our ATP is improved by 30% and 72% compared to [32] and [33]. Moreover, in [40], Ni et al. present a BRAM-free NTT design. The authors propose the FIFO-based ping-pong method to reduce memory footprint. Their NTT executes NTT/INTT/PWM in 456/456/265 CCs, and slower than us. Our NMI-NTT has ATP better with an improvement of 21%.

Next, in Tab V, we report our hardware implementation results of Kyber.CCAKEM and compare them with studies reported to date. Reference [4] presents a state-of-the-art compact design. Xing et al. chose a dual-butterfly configuration similar to our NMI-NTT design. However, their design employs considerable FIFOs for data storage within and between modules. Their SHA3 module has faster hashing times. Nonetheless, executing NTT and INTT requires additional CCs for data writing into RAM and compression. Therefore, their design requires 448 + 64 CCs and 448 + 128 CCs for NTT and INTT, respectively. It follows that their execution data flow is significantly less efficient. In contrast, we implement comprehensive optimizations for the execution data flow. Hence, our design has fewer total execution CCs than [4]. About BRAM usage, we use more than 0.5 BRAMs because [4] also uses large FIFOs to store ciphertext (ct) instead. The reported results show that our design performs better in terms of speed and area. Our ATPs have improved by approximately 25% compared to theirs across all security levels.

In [13], the authors proposed a CRYSTALS-Kyber architecture with a mix-radix2/4 butterfly configuration and a standard SHA3 core. Consequently, their NTT execution time is twice as fast as ours. The results show that their CRYSTALS-Kyber architecture achieves faster key generation, encapsulation, and decapsulation. This design is reported to consume resources totaling 2.2k SLICEs, 5 BRAMs, and 4 DSPs, corresponding to an ENS of 3.6k for three security levels. It is noted that [13] did not analyze and report the resources allocated for storing the public key (pk) and ciphertext (ct). That means the 5 BRAMs are divided into 4 BRAMs for polynomial multiplication and 1 BRAM to store the secret key (sk). Our design performs better in terms of ATP, corresponding

TABLE IV
IMPLEMENTATION RESULTS OF THE PROPOSED NMI-NTT AND COMPARISON WITH PREVIOUS WORK

Work	Freq [MHz]	LUT	FFs	SLICEs	BRAMs	DSPs	ENS[1]	Cycles			ATP[2]
								NTT	INTT	PWM	
[38]	190	948	352	281	2.5	1	881	904	904	3359	1593 (+71%)
[39]	222	801	717	290	2	4	1090	324	324	N/A	706 (+34%)
[11]	115	737	290	221	4	6	1621	474	602	1289	1744 (+73%)
[4]	161	1579	1058	527	3	2	1327	512	576	256	1444 (+68%)
[7]	229	880	999	345	1.5	2	845	448	448	256	757 (+38%)
[13]	200	1740	643	575	4	4	1775	225	225	N/A	799 (+42%)
[32]	200	784	441	259	3	2	1059	313	313	N/A	663 (+30%)
[33]	239	1603	2004	651	4.5	6	2151	384	384	132	1652 (+72%)
[40]	300	1154	1031	445	0	2	645	456	456	265	588 (+21%)
[41]	303	1170	1164	416	2	4	1216	235	235	138	572 (+19%)
[25]	250	1045	638	341	3.5	11	2141	933	933	265	3995 (+88%)
This work	**227**	**1005**	**599**	**319**	**0**	**2**	**519**	**448**	**448**	**256**	**465 (0%)**

[1] The Equivalent Number of Slices ENS = LUTs/4 + FFs/8 + BRAMs×200 + DSPs×100.
[2] The Area-Time Product (ATP) is calculated as Total Clock Cycles (NTT + INTT) × ENS.

TABLE V
IMPLEMENTATION AND COMPARISON RESULTS FOR DIFFERENT KYBER.CCAKEM INSTANCES

Work	Freq [MHz]	Cycle Count			Total time [μs]	Area					ENS[1] [k]	ATP[2]
		Keygen [kCCs]	Encap. [kCCs]	Decap. [kCCs]		LUTs [k]	FFs [k]	SLICEs [k]	BRAMs	DSPs		
Kyber.CCAKEM, Security level 1 (Kyber512)												
[4]	161	3.8	5.1	6.7	95.2	7.4	4.6	2.1	3	2	2.9	45.2 (+26%)
[13]	200	2.4	3.0	4.2	48.3	7.3	3.2	2.2	5	4	3.6	35.0 (+5%)
[33]	213	1.7	2.4	3.5	35.7	12.0	14.0	4.7	9	12	7.7	58.9 (+43%)
[32]	200	3.0	3.9	5.0	66.5	6.9	3.3	2.0	3	2	2.8	34.4 (+3%)
[7]	220	2.1	3.3	4.5	44.8	9.3	8.2	3.4	6	4	5.0	49.1 (+32%)
[39]	200	1.9	2.4	3.7	40.4	10.5	9.8	3.5	13	8	6.9	55.6 (+40%)
[11]	115	4.0	7.0	10.0	182.7	18.0	5.0	5.0	6	15	7.7	161.7 (+79%)
[5]	208	1.1	1.5	2.1	22.6	16.1	13.8	5.3	0	2	5.5	25.7 (-30%)
This work	**185**	**3.3**	**4.5**	**6.1**	**75.1**	**5.5**	**3.4**	**1.5**	**3.5**	**2**	**2.4**	**33.4 (0%)**
Kyber.CCAKEM, Security level 3 (Kyber768)												
[4]	161	6.3	7.9	10.0	149.1	7.4	4.6	2.1	3	2	2.9	70.2 (+25%)
[13]	200	4.2	5.0	7.0	81.3	7.3	3.2	2.2	5	4	3.6	59.0 (+11%)
[33]	210	2.1	3.4	4.5	47.6	14.0	17.0	5.6	13	18	10.0	100.3 (+48%)
[32]	200	5.1	6.0	7.9	96.5	6.9	3.3	2.0	3	2	2.8	54.8 (+4%)
[7]	220	2.7	3.9	5.0	52.9	10.4	9.5	3.8	8.5	6	6.1	70.8 (+26%)
[39]	200	2.6	3.3	4.8	53.6	11.8	10.4	4.0	14	12	8.0	85.1 (+38%)
[11]	115	7.0	10.0	14.0	269.6	16.0	6.0	4.0	9	16	7.4	229.4 (77%)
[5]	208	1.7	2.4	3.0	34.1	16.8	13.7	5.6	0	2	5.8	41.2 (-28%)
This work	**185**	**5.6**	**7.1**	**9.2**	**118.4**	**5.5**	**3.4**	**1.5**	**3.5**	**2**	**2.4**	**52.6 (0%)**
Kyber.CCAKEM, Security level 5 (Kyber1024)												
[4]	161	9.4	11.3	13.9	212.3	7.4	4.6	2.1	3	2	2.9	100.3 (+25%)
[13]	200	6.8	8.0	10.2	126.1	7.3	3.2	2.2	5	4	3.6	90.0 (+16%)
[33]	204	3.9	4.5	5.6	68.6	16.0	19.0	6.4	15	24	11.8	164.8 (+54%)
[32]	200	8.01	9.03	11.2	142.6	6.9	3.3	2.0	3	2	2.8	80.2 (+6%)
[7]	220	3.6	4.8	6.0	65.2	11.5	10.6	4.2	10.5	8	7.1	102.6 (+26%)
[39]	185	3.5	4.1	6.3	74.8	13.3	11.6	4.6	16	16	9.4	130.6 (+42%)
[11]	115	10.0	14.0	18.0	365.2	16.0	6.0	5.0	12	17	9.1	382.2 (+80%)
[37]	159	7.8	8.4	10.5	167.9	7.9	3.9	2.3	16	4	5.9	157.5 (+52%)
[5]	208	2.7	3.4	4.1	49.0	17.8	14.0	6.0	0	2	6.2	63.5 (-19%)
This work	**185**	**8.5**	**10.1**	**12.9**	**170.3**	**5.5**	**3.4**	**1.5**	**3.5**	**2**	**2.4**	**75.6 (0%)**

[1] The Equivalent Number of Slices ENS = LUTs/4 + FFs/8 + BRAMs×200 + DSPs×100.
[2] The Area-Time Product (ATP) is calculated as Total Clock Cycles (Keygen + Encapsulation + Decapsulation) × ENS.

to 5%, 11%, and 16% for Kyber512, Kyber768, and Kyber1024.

Studies [7], [11], [37], [39] utilize high-level parallel BU configurations to implement CRYSTALS-Kyber. In particular, [7] proposed parallel k-polynomials execution to balance speed and memory usage. However, increasing the parallel level of memory-based iterative NTT increases BRAM utilization exponentially. Moreover, these designs also use considerable FIFOs for data storage between modules. Therefore, their design efficiency is significantly less effective. Our design outperforms hardware efficiency ATP by 26%, 42%, 52%, and 80% compared to [7], [11], [37], and [39] at security level three, respectively.

Reference [32] presents a compact CRYSTALS-Kyber design based on a split-radix NTT. Similar to [13], the storage of (pk, ct) is also not addressed and reported in [32]. Despite incomplete hardware resource reporting, our design performs slightly better across all three security levels. Besides, Li et al. proposed a split-radix NTT design for high-speed CRYSTALS-Kyber implementation [33]. Their memory

strategy is quite efficient and scalable for k polynomials. Like [7] and [33] performs NTT in parallel at the polynomial level instead of the stage level. However, their data flow for NTT/INTT is ineffective, resulting in slow execution times. Our design outperforms theirs in hardware efficiency ATP by 43%, 48%, and 54% for Kyber512, Kyber768, and Kyber1024, respectively.

In [5], Ni et al. designed a standard MDC pipeline NTT architecture for CRYSTALS-Kyber. Their MDC-NTT requires six reordering units with a fixed configuration for each NTT/INTT stage. Our RU design has better area efficiency due to the flexible delay configuration for all NTT and INTT stages. Additionally, the authors used FIFOs for data memory between modules. Hence, their design consumes a considerable amount of hardware resources. However, polynomial multiplication based on NTT is the most time-consuming part of CRYSTALS-Kyber implementation. Moreover, the pipelined NTT architecture has a simple control pattern and requires no additional memory. As a result, our hardware efficiency ATP is less effective than theirs by 30%, 28%, and 19% for Kyber512, Kyber768, and Kyber1024, respectively.

V. Conclusion

This paper proposes a lightweight CRYSTALS-Kyber architecture for constrained IoT devices. To achieve low hardware cost efficiency, we optimize each core operation of CRYSTALS-Kyber. First, we design a novel compact SHA3 module that performs hashing operations on a single 1600-state buffer. Compared to existing designs, our SHA3 does not require costly FIFOs for input and output data, saving hardware costs by 29% to 60%. Next, we propose a novel non-memory-based iterative NTT (NMI-NTT) for polynomial multiplication in CRYSTALS-Kyber. Our NMI-NTT eliminates the need for additional memory overhead and complex access patterns in the iterative NTT. Implementation results demonstrate that NMI-NTT outperforms reported-to-date iterative NTT designs in ATP by 19% to 88%. Finally, we optimize data flow processing, significantly reducing execution time for Kyber.CPA.KeyGen, Kyber.CPA.Enc, and Kyber.CPA.Dec. FPGA implementation reports show that our CRYSTALS-Kyber architectures achieve the lowest hardware footprint in the literature. Additionally, our design supports all three security levels, with hardware efficiency in terms of ATP improved by up to 80% compared to other designs. The optimizations proposed in this paper can serve as a case study for the lightweight implementation of other PQC algorithms.

In addition, our implementations are constant-time and hence can be resistant to timing attacks, a type of side-channel attack (SCA) [42]. Furthermore, there are many other types of SCAs, and developing countermeasures against such attacks is necessary [1]. In particular, studies [43], [44], [45] have proposed fault detection architectures to counter SCA-fault attacks with low hardware costs. Therefore, our future work will focus on implementing countermeasures against SCAs suitable for lightweight PQC implementations.

References

[1] G. Alagic, "Status report on the third round of the NIST post-quantum cryptography standardization process," NIST, Gaithersburg, MD, USA, Tech. Rep. IR 8413, 8413.

[2] D. Atkins, "Requirements for post-quantum cryptography on embedded devices in the IoT," in *Proc. NIST PQC Standardization Conf.*, Jun. 2021, pp. 7–9.

[3] Nat. Inst. Standards Technol. (Aug. 2015). *SHA-3 Standard: Permutationbased Hash and Extendable-Output Functions*. [Online]. Available: https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.202.pdf

[4] Y. Xing and S. Li, "A compact hardware implementation of CCA-secure key exchange mechanism CRYSTALS-KYBER on FPGA," *IACR Trans. Cryptograph. Hardw. Embedded Syst.*, vol. 2021, no. 2, pp. 328–356, Feb. 2021.

[5] Z. Ni, A. Khalid, D.-E-S. Kundi, M. O'Neill, and W. Liu, "HPKA: A high-performance CRYSTALS-Kyber accelerator exploring efficient pipelining," *IEEE Trans. Comput.*, vol. 72, no. 12, pp. 3340–3353, Jul. 2023.

[6] A. Dolmeta, M. Martina, and G. Masera, "Hardware architecture for CRYSTALS-Kyber post-quantum cryptographic SHA-3 primitives," in *Proc. 18th Conf. Ph.D Res. Microelectron. Electron. (PRIME)*, Jun. 2023, pp. 209–212.

[7] V. B. Dang, K. Mohajerani, and K. Gaj, "High-speed hardware architectures and FPGA benchmarking of CRYSTALS-Kyber, NTRU, and saber," *IEEE Trans. Comput.*, vol. 72, no. 2, pp. 306–320, Feb. 2023.

[8] G. Bertoni, J. Daemen, M. Peeters, and G. V. Assche. (Sep. 2011). *Keccak Implementation Overview-3.1*. [Online]. Available: https://keccak.team/obsolete/Keccak-implementation-3.1.pdf

[9] Y. Yang, D. He, N. Kumar, and S. Zeadally, "Compact hardware implementation of a SHA-3 core for wireless body sensor networks," *IEEE Access*, vol. 6, pp. 40128–40136, 2018.

[10] T. Fritzmann, G. Sigl, and J. Sepúlveda, "RISQ-V: Tightly coupled RISC-V accelerators for post-quantum cryptography," *IACR Trans. Cryptograph. Hardw. Embedded Syst.*, vol. 2020, no. 4, pp. 239–280, Aug. 2020.

[11] M. Bisheh-Niasar, R. Azarderakhsh, and M. Mozaffari-Kermani, "Instruction-set accelerated implementation of CRYSTALS-Kyber," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 68, no. 11, pp. 4648–4659, Nov. 2021.

[12] Y. Zhao, R. Xie, G. Xin, and J. Han, "A high-performance domain-specific processor with matrix extension of RISC-V for module-LWE applications," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 69, no. 7, pp. 2871–2884, Jul. 2022.

[13] W. Guo and S. Li, "Highly-efficient hardware architecture for CRYSTALS-Kyber with a novel conflict-free memory access pattern," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 70, no. 11, pp. 4505–4515, Nov. 2023.

[14] J. W. Cooley and J. W. Tukey, "An algorithm for the machine calculation of complex Fourier series," *Math. Comput.*, vol. 19, no. 90, pp. 297–301, Apr. 1965.

[15] W. M. Gentleman and G. Sande, "Fast Fourier transforms: For fun and profit," in *Proc. Fall Joint Comp. Conf. (AFIPS)*, Nov. 1966, pp. 563–578.

[16] U. Banerjee, T. S. Ukyab, and A. P. Chandrakasan, "Sapphire: A configurable crypto-processor for post-quantum lattice-based protocols," 2019, *arXiv:1910.07557*.

[17] S. S. Roy, F. Vercauteren, N. Mentens, D. D. Chen, and I. Verbauwhede, "Compact ring-LWE cryptoprocessor," in *Proc. Cryptol. Hardw. Embedded Syst. (CHES)*. Berlin, Germany: Springer, Sep. 2014, pp. 371–391.

[18] T. Pöppelmann, T. Oder, and T. Güneysu, "High-performance ideal lattice-based cryptography on 8-bit ATxmega microcontrollers," in *Proc. Int. Conf. Cryptol. Info. Secu. Latin Amer. (LATINCRYPT)*, Aug. 2015, pp. 346–365.

[19] Z. Ye, R. C. C. Cheung, and K. Huang, "PipeNTT: A pipelined number theoretic transform architecture," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 69, no. 10, pp. 4068–4072, Oct. 2022.

[20] F. Hirner, A. C. Mert, and S. S. Roy, "Proteus: A pipelined NTT architecture generator," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 32, no. 7, pp. 1228–1238, Jul. 2024.

[21] D.-e.-S. Kundi, Y. Zhang, C. Wang, A. Khalid, M. O'Neill, and W. Liu, "Ultra high-speed polynomial multiplications for lattice-based cryptography on FPGAs," *IEEE Trans. Emerg. Topics Comput.*, vol. 10, no. 4, pp. 1993–2005, Oct. 2022.

[22] T.-H. Nguyen, B. Kieu-Do-Nguyen, C.-K. Pham, and T.-T. Hoang, "High-speed NTT accelerator for CRYSTAL-Kyber and CRYSTAL-Dilithium," *IEEE Access*, vol. 12, pp. 34918–34930, 2024.

[23] K.-F. Xia, B. Wu, T. Xiong, and T.-C. Ye, "A memory-based FFT processor design with generalized efficient conflict-free address schemes," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 25, no. 6, pp. 1919–1929, Jun. 2017.

[24] A. Aikata, A. C. Mert, M. Imran, S. Pagliarini, and S. S. Roy, "KaLi: A crystal for post-quantum security using Kyber and Dilithium," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 70, no. 2, pp. 747–758, Feb. 2023.

[25] S. D. Matteo, I. Sarno, and S. Saponara, "CRYPHTOR: A memory-unified NTT-based hardware accelerator for post-quantum CRYSTALS algorithms," *IEEE Access*, vol. 12, pp. 25501–25511, 2024.

[26] A. C. Mert, E. Karabulut, E. Öztürk, E. Savas, and A. Aysu, "An extensive study of flexible design methods for the number theoretic transform," *IEEE Trans. Comput.*, vol. 71, no. 11, pp. 2829–2843, Nov. 2022.

[27] X. Chen, B. Yang, S. Yin, S. Wei, and L. Liu, "CFNTT: Scalable Radix-2/4 NTT multiplication architecture with an efficient conflict-free memory mapping scheme," *IACR Trans. Cryptograph. Hardw. Embedded Syst.*, vol. 2022, no. 1, pp. 94–126, Nov. 2021.

[28] X. Hu, J. Tian, M. Li, and Z. Wang, "AC-PM: An area-efficient and configurable polynomial multiplier for lattice based cryptography," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 70, no. 2, pp. 719–732, Feb. 2023.

[29] J. Mu et al., "Scalable and conflict-free NTT hardware accelerator design: Methodology, proof, and implementation," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 42, no. 5, pp. 1504–1517, May 2023.

[30] Y. Zhao, X. Liu, Y. Hu, and H. Xiao, "Design of an efficient NTT/INTT architecture with low-complex memory mapping scheme," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 71, no. 1, pp. 400–404, Jan. 2024.

[31] S.-H. Liu, C.-Y. Kuo, Y.-N. Mo, and T. Su, "An area-efficient, conflict-free, and configurable architecture for accelerating NTT/INTT," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 32, no. 3, pp. 519–529, Mar. 2024.

[32] W. Guo and S. Li, "Split-radix based compact hardware architecture for CRYSTALS-Kyber," *IEEE Trans. Comput.*, vol. 73, no. 1, pp. 97–108, Jan. 2024.

[33] G. Li, D. Chen, G. Mao, W. Dai, A. I. Sanka, and R. C. C. Cheung, "Algorithm-hardware co-design of split-radix discrete Galois transformation for KyberKEM," *IEEE Trans. Emerg. Topics Comput.*, vol. 11, no. 4, pp. 824–838, May 2023.

[34] R. Avanzi et al. (Jan. 2021). *CRYSTALS-Kyber: Algorithm Specifications and Supporting Documentation*. [Online]. Available: https://pq-crystals.org/kyber/data/kyber-specification-round3-20210131.pdf

[35] V. Lyubashevsky, D. Micciancio, C. Peikert, and A. Rosen, "SWIFFT: A modest proposal for FFT hashing," in *Proc. Fast Softw. Encryp. (FSE)*, Feb. 2008, pp. 54–72.

[36] T.-H. Nguyen, C.-K. Pham, and T.-T. Hoang, "A high-efficiency modular multiplication digital signal processing for lattice-based post-quantum cryptography," *Cryptography*, vol. 7, no. 4, p. 46, Sep. 2023.

[37] W. Guo, S. Li, and L. Kong, "An efficient implementation of KYBER," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 69, no. 3, pp. 1562–1566, Mar. 2022.

[38] F. Yaman, A. C. Mert, E. Öztürk, and E. Savas, "A hardware accelerator for polynomial multiplication operation of CRYSTALS-KYBER PQC scheme," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, Feb. 2021, pp. 1020–1025.

[39] M. Bisheh-Niasar, R. Azarderakhsh, and M. Mozaffari-Kermani, "High-speed NTT-based polynomial multiplication accelerator for post-quantum cryptography," in *Proc. IEEE 28th Symp. Comput. Arithmetic (ARITH)*, Jun. 2021, pp. 94–101.

[40] Z. Ni, A. Khalid, W. Liu, and M. O'Neill, "Towards a lightweight CRYSTALS-Kyber in FPGAs: An ultra-lightweight BRAM-free NTT core," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, May 2023, pp. 1–5.

[41] M. Li, J. Tian, X. Hu, and Z. Wang, "Reconfigurable and high-efficiency polynomial multiplication accelerator for CRYSTALS-Kyber," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 42, no. 8, pp. 2540–2551, Dec. 2022.

[42] P. C. Kocher, "Timing attacks on implementations of Diffie–Hellman, RSA, DSS, and other systems," in *Proc. Annual Int. Cryptol. Conf. Adv. Cryptol. (CRYPTO)*, Aug. 1996, pp. 104–113.

[43] A. Sarker, M. M. Kermani, and R. Azarderakhsh, "Fault detection architectures for inverted binary ring-LWE construction benchmarked on FPGA," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 68, no. 4, pp. 1403–1407, Apr. 2021.

[44] A. Sarker, A. C. Canto, M. M. Kermani, and R. Azarderakhsh, "Error detection architectures for hardware/software co-design approaches of number-theoretic transform," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 42, no. 7, pp. 2418–2422, Jul. 2023.

[45] S. Khan et al., "Efficient, error-resistant NTT architectures for CRYSTALS-Kyber FPGA accelerators," in *Proc. IFIP/IEEE 31st Int. Conf. Very Large Scale Integr. (VLSI-SoC)*, Oct. 2023, pp. 1–6.

Trong-Hung Nguyen (Graduate Student Member, IEEE) received the B.Sc. degree in control engineering and automation from Hanoi University of Science and Technology (HUST), Hanoi, Vietnam, in 2014, and the M.S. degree in electronic engineering from The University of Electro-Communications (UEC), Tokyo, Japan, in 2022, where he is currently pursuing the Ph.D. degree in information and network engineering. His research interests include digital signal processing, hardware accelerators, and post-quantum cybersecurity.

Duc-Thuan Dam (Graduate Student Member, IEEE) received the B.Sc. degree in electrical and electronic engineering and the M.S. degree in electronic engineering from Le Quy Don Technical University, Hanoi, Vietnam, in 2013 and 2019, respectively. He is currently pursuing the Ph.D. degree in information and network engineering with The University of Electro-Communications (UEC), Tokyo, Japan. His research interests include forward error correction codes, post-quantum cryptography, hardware implementation, and RISC-V.

Phuc-Phan Duong (Graduate Student Member, IEEE) received the B.Sc. and M.S. degrees from the Department of Electronics and Telecommunications, Posts and Telecommunications Institute of Technology (PTIT), Hanoi, Vietnam, in 2011 and 2014, respectively. He is currently pursuing the Ph.D. degree in information and network engineering with The University of Electro-Communications (UEC), Tokyo, Japan. From 2013 to 2023, he was a Lecturer Assistant with the Academy of Cryptography Techniques. His research interests include cryptography, embedded systems design, and hardware security.

Binh Kieu-Do-Nguyen (Graduate Student Member, IEEE) received the B.Sc. and M.S. degrees from the Department of Computer Science and Engineering, Ho Chi Minh City University of Technology (HCMUT), Ho Chi Minh City, Vietnam, in 2017 and 2019, respectively. He is currently pursuing the Ph.D. degree in information and network engineering with The University of Electro-Communications (UEC), Tokyo, Japan. From 2017 to 2021, he was a Lecturer Assistant with HCMUT. Since 2022, he has been a Research Assistant with UEC. His research interests include computer architecture, hardware security, and digital systems design.

Cong-Kha Pham (Senior Member, IEEE) received the B.S., M.S., and Ph.D. degrees in electronics engineering from Sophia University, Tokyo, Japan. He is currently a Professor with the Department of Information and Network Engineering, The University of Electro-Communications (UEC), Tokyo. The University of Electro-Communications Integrated Circuit Design Laboratory (Pham Lab) educates the design, implementation, and evaluation of hardware systems and VLSI, aims to design a system-on-chip by integrating various information processing hardware, and develops a high-performance computational circuit realized with a small number of elements. His research interests include hardware system design implementation by FPGA and integrated circuits.

Trong-Thuc Hoang (Member, IEEE) received the B.Sc. and M.S. degrees in electronic engineering from Ho Chi Minh City University of Science (HCMUS), Ho Chi Minh City, Vietnam, in 2012 and 2017, respectively, and the Ph.D. degree in engineering from The University of Electro-Communications (UEC), Tokyo, Japan, in 2022. From 2012 to 2017, he was a Lecturer Assistant with HCMUS. From 2019 to 2020, he was a Research Assistant with UEC. From 2019 to 2022, he was a Research Assistant with the Cyber-Physical Security Research Center (CPSEC), National Institute of Advanced Industrial Science and Technology (AIST), Tokyo. Since April 2022, he has been an Assistant Professor with the Department of Computer and Network Engineering, UEC. His research interests include digital signal processing, computer architecture, cyber-security, and ultra-low power systems-on-a-chip.